説明資料 — 自然言語指示で動く差動二輪ロボット（ROS 2 × Gazebo × LLM）

Physical AI 応用1 最終課題 ／ omnicampus アカウント名: yonezawakengo
コード: https://github.com/ken617/pai-final-llm-robot ／ 動画: （YouTube URL を記入）

1. 概要

ROS 2 Humble と Gazebo Fortress 上の差動二輪ロボットに対し、「赤い箱まで行って、そのあと青い箱へ」といった 自然言語で指示を与えると、LLM（大規模言語モデル）が行動計画を JSON で生成し、ロボットがオドメトリに基づく 閉ループ制御で自律的に実行するシステムを構築した。

2. 目的

- 講義で扱った ROS 2 と Gazebo の連携（トピック・ブリッジ・launch）を、実際に動く形で習得する。
- 「人が機械の言葉（キー入力・座標）に合わせる」操作を、「機械が人の言葉に合わせる」操作へ置き換え、 LLM を推論器として実ロボット制御の入口に組み込む設計を試す。
- LLM 出力をそのまま信じず、検証・クリップしてから実行する「安全なAI制御」の型を実装する。

3. ベースコードと変更点

ベース: Qiita「ROS2とGazeboの連携によるロボット操作」（N622, 初級サンプル）

項目	ベース記事	本課題
操作方法	キーボード（w/a/s/d）→ Twist	**自然言語 → LLM → 行動計画(JSON) → Twist**
制御	開ループ（押している間だけ速度指令）	**オドメトリを用いた閉ループ P 制御**（goto / move / turn）
ブリッジ	/cmd_vel のみ (ROS→GZ)	/cmd_vel に加え **/odom, /clock (GZ→ROS)** を追加
ワールド	ロボットと地面のみ	**ランドマーク（赤・青・緑の箱）** を配置し、LLM に座標を知識として与える
実装言語	C++（key_event パッケージ依存）	**Python (rclpy, ament_python)**。外部依存なし（標準ライブラリで LLM API 呼び出し）
実行環境	Ubuntu 22.04 実機	**Docker（noVNC）** で Mac/Windows でも再現可能
AI 要素	なし	**LLM 推論（学習なし）**。OpenAI 互換 API / ローカル LLM (Ollama) を切替可

4. システム構成

```
[say CLI] --/user_command--> [llm_commander] --/robot_plan(JSON)--> [plan_executor]
                                (LLM API 呼び出し)                        |      ^
                                                                  /cmd_vel|      | /odom
                                                                          v      |
                                                          [ros_gz_bridge] <--> [Gazebo DiffDrive]
```

- `llm_commander`: システムプロンプトに「使えるアクション」「座標系」「ランドマーク座標」「安全ルール」を記述。 LLM の出力から JSON 配列を抽出し、`validate_plan()` で型チェックと数値のクリップ（直進 ≤3 m、回転 ≤360°、待機 ≤10 s）を行う。
- `plan_executor`: アクションキューを状態機械で順次実行。goto は「向きのずれが大きければ回頭、小さければ前進しながら補正」の 2 段階 P 制御。
- `plan_status` トピックに進捗を出し、動画のテロップ・デバッグに利用。

5. 工夫した点

1. **LLM の役割を「計画」に限定**し、低レベル制御は決定論的な P 制御に任せた。LLM を毎ステップ呼ぶ方式に比べ、 応答遅延・コスト・不安定さを排除でき、実機でも使える構造になっている。
2. **出力検証（ガードレール）**: LLM の JSON を検証・クリップし、未知のアクションは拒否。不可能な指示には `say: 実行できません` を返させ、ロボットを動かさない。
3. **ランドマーク知識のプロンプト注入**:「赤い箱」の座標をプロンプトに書くことで、 視覚認識なしに「〜の前まで行く」が成立する（箱の手前 0.6 m に停止）。
4. **再現性**: Dockerfile + compose で環境を固定し、.env で LLM バックエンドを切替可能にした（API 不要のローカル LLM でも動作）。
5. **ROS なしで回るテスト**を用意した。rclpy をスタブ化し、制御ノードを運動学モデルで閉ループ実行する `tests/test_logic.py` と、LLM の計画生成だけを確認する `tests/check_llm.py`。Gazebo を立ち上げる前にロジックの不具合を潰せる。
6. ベース記事で不整合だったキー定義・ファイル名を整理し、Python 1 パッケージに統合した。

6. 結果

6.1 シミュレーションテスト（ROS なし、運動学モデル）

テスト	結果
LLM 出力の解釈・クリップ（コードフェンス除去、直進 99 m→3 m、回転 −720°→−360°、未知アクション拒否）	合格
goto (2.4, 0) → goto (0, 2.4) の連続実行	合格（最終誤差 0.12 m、19.1 s）
左 90° 回転 → 2 m 直進	合格（到達 (0.07, 1.98) m、方位 87.9°）
後退 1 m → 2 s 待機 → 発話	合格（x = −0.98 m）
速度上限（0.4 m/s, 0.9 rad/s）の遵守	合格

6.2 Gazebo 上での実行（動画）

（動画撮影後に記入: 試した指示文、LLM が生成した計画、成功/失敗、所要時間）

- 「赤い箱まで行って」→
- 「青い箱に寄ってから、緑の箱の前で3秒待って、スタート地点に戻って」→
- 「空を飛んで」→ `実行できません` が返りロボットは動かない（想定）

7. 実用性

受付ロボット・病院や介護施設内の搬送ロボットなど、専門家でない現場スタッフが 「ナースステーションまで行って」と言葉で指示する用途に直結する。 LLM 部分と制御部分を分離しているため、実機では Nav2 の goto に差し替えるだけで移行できる。

8. 反省点・今後の課題

- オドメトリのみの推定なので、長距離走行では誤差が蓄積する（LiDAR/カメラによる位置補正が必要）。
- ランドマーク座標を事前知識として与えており、未知環境では VLM による物体認識が必要。
- （実装中に起きた問題・解決策をここに記入）

9. 使用技術

ROS 2 Humble, Gazebo Fortress (Ignition), ros_gz_bridge, rclpy, Docker, OpenAI 互換 API（gpt-4o-mini または Ollama qwen2.5:7b）

参考

- ベース記事: https://qiita.com/N622/items/9d89f77d85d9da0af29e
- Gazebo 公式チュートリアル "Moving the robot", ros_gz